Automated Inventory Data Entry through Intelligent Character and Document Structure Recognition

By

Arifur Rahman ID: 20234103222

Meer Mushfiqur Rahman ID: 20234103240

Fatema Akter Akhi ID: 20234103212

Sheikh Ramisha Zubaydah ID: 20234103236

Supervisor

Md. Mijanur Rahman

Assistant Professor, CSE, BUBT

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Science in

Computer Science and Engineering

Department of Computer Science and Engineering

Bangladesh University of Business and Technology

July 2026

Declaration

We do hereby declare that the research works presented in this thesis entitled, "Automated Inventory Data Entry through Intelligent Character and Document Structure Recognition" are the results of our works. We further declare that the thesis/report has been compiled and written by us, and no part of this thesis/report has been submitted elsewhere for the requirements of any degree, award, diploma, or any other purposes except for publications. The materials that are obtained from other sources are duly acknowledged in this thesis.

Signature of Authors

Arifur Rahman

ID: 20234103222

Meer Mushfiqur Rahman

ID: 20234103240

Fatema Akter Akhi

ID: 20234103212

Sheikh Ramisha Zubaydah

ID: 20234103236

Approval

It is hereby certified that the research work presented in this thesis entitled, "Automated Inventory Data Entry through Intelligent Character and Document Structure Recognition," is the result of the original work carried out by Arifur Rahman, Meer Mushfiqur Rahman, Fatema Akter Akhi, and Sheikh Ramisha Zubaydah. Furthermore, it is certified that this thesis/report satisfies the requirements and standards for the degree of Bachelor of Science in Computer Science and Engineering.

Md. Mijanur Rahman — **Supervisor**
Assistant Professor, CSE — Capstone Defense Board
BUBT, Dhaka

Md. Masudul Islam — **Course Teacher**
Assistant Professor, CSE — Capstone Defense Board
BUBT, Dhaka

Prof. Dr. Md. Ahsan Habib — **Convener**
Chairman, CSE — Capstone Defense Board
BUBT, Dhaka

Department of Computer Science and Engineering, BUBT, Dhaka.

Dedication

We dedicate this research to our loving parents, whose unwavering love, encouragement, sacrifices, and constant support have been the foundation of our academic journey. Their guidance and belief in our abilities have inspired us to overcome challenges and pursue excellence. This work is also dedicated to our families, teachers, and well-wishers for their continuous motivation and encouragement throughout the completion of this thesis.

Acknowledgement

We would like to express our heartfelt gratitude to the almighty Allah who offered our family and us kind care throughout this journey. We are deeply thankful to the Bangladesh University of Business and Technology (BUBT) for providing us with such a wonderful environment to pursue our research/project. We would like to express our sincere gratitude to our supervisor, Md. Mijanur Rahman, Assistant Professor, Department of CSE, BUBT. We have completed our research with his help. We found the research area, topic, and problem with his suggestions. He guided us with our study and supplied us with many research papers and academic resources in this area. He is patient and responsible. When we had questions and needed his help, he would always find time to meet and discuss with us no matter how busy he was. We would also like to thank our course teacher, Md. Masudul Islam, Assistant Professor, Department of CSE, BUBT, for his guidance throughout this course. We also want to give thanks to the Chairman and all our teachers in the Department of Computer Science and Engineering for providing a solid background for our studies and research thereafter. They have been a great source of inspiration to us, and we thank them from the bottom of our hearts. We would also like to acknowledge our team members for supporting each other and be grateful to our university for providing this opportunity for us.

Abstract

Small and medium retail businesses generally keep their purchase and sale records on paper: printed invoices, thermal receipts, and handwritten chits, each supplier using its own format. Because manually entering this information into an inventory or ERP system is slow, the digitization step is often inconsistent, leaving businesses without a reliable, structured record of their transactions. This report proposes a mobile-to-cloud document digitization gateway that converts supplier invoices, receipts, and other inventory documents into a standardized JSON output, delivered through an API that an inventory or ERP system can call to automate what is currently manual data entry. The main technical component is character and document structure recognition: reading the text of a scanned document and identifying its structural role, such as a purchase order, a customer copy, or a line-item table, after which a cloud-hosted language model extracts the content into structured line items under a fixed schema. Since language models can produce incorrect or fabricated values, the system includes arithmetic self-validation and confidence scoring so that output is checked before it is finalized. Reconciling item names that differ across suppliers is left to the consuming inventory or ERP system. This report covers the introduction, background study, and proposed research methodology of the project. Chapter 1 states the problem along with the project's objectives, motivation, significance, and contributions. Chapter 2 reviews the relevant literature across four thematic areas and compares the project against existing commercial systems. Chapter 3 presents the proposed methodology, including the system architecture, module design, corpus collection plan, and evaluation framework.

Keywords: *Document Digitization; Optical Character Recognition; Document Structure Recognition; Large Language Models; Schema-Constrained Extraction; Idempotent API Design.*

List of Figures

3.1 Development methodology: the phases of the iterative, evaluation-driven lifecycle and the feedback loop from evaluation back into the API contract and extraction design. 21

3.2 Proposed system architecture: mobile capture, cloud-hosted recognition and extraction pipeline with arithmetic self-validation, and delivery to an authorized ERP or inventory system via the digitization API. 21

3.3 Detailed pipeline diagram: per-stage inputs and outputs from captured image through to JSON response, showing where arithmetic self-validation and confidence flagging attach. 23

3.4 Document-type taxonomy: the four target classes with the structural and lexical cues that distinguish them, and the inventory implication of each. 31

3.5 API sequence diagram: capture, submit (with idempotency key), process, result retrieval by the consuming ERP, including the duplicate-submission path returning the original result without reprocessing. . . 34

3.6 Entity-relationship diagram: the Submission record, alongside the inventory and catalog entities deliberately excluded from the gateway's scope. 34

List of Tables

1 Complex Engineering Problem attributes 7

2 Complex Engineering Activity attributes 8

3 Consolidated literature review by theme 14

4 Comparison against existing commercial systems 17

5 Target corpus composition (provisional) 24

6 Proposed technology stack (provisional) 25

7 Open algorithm and model decisions 28

8 Target JSON output schema (provisional; to be revised after corpus collection) 32

List of Abbreviations and Acronyms

OCR Optical Character Recognition

LLM Large Language Model

ERP Enterprise Resource Planning

API Application Programming Interface

JSON JavaScript Object Notation

NLP Natural Language Processing

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Problem Background 2

1.3 Research Objectives 3

1.4 Motivations 4

1.5 Significance of the Research 5

1.6 Research Contributions 6

1.7 Complex Engineering Analysis 6

1.7.1 Complex Engineering Problems 7

1.7.2 Complex Engineering Activities 7

1.8 Thesis/Project Organization 8

2 Background Study **10**

2.1 Literature Review 10

2.1.1 Character and Document Structure Recognition 10

2.1.2 Schema-Constrained Structured Output from Language Models 11

2.1.3 Hallucination Mitigation, Self-Verification, and Confidence Calibration 12

2.1.4 Document-Gateway Architecture and API Idempotency 13

2.1.5 Summary of the Literature 14

2.2 Problem Analysis 14

2.2.1 Decomposition of the Technical Problem 14

2.2.2 Analysis of Existing Systems 15

2.2.3 Consolidated Research Gap 16

3 Research Methodology **19**

3.1 Proposed Framework 19

3.2 Data Set Analysis / Software Project Requirement 22

3.2.1 Data Set Analysis 22

3.2.2 Software Project Requirement 24
3.3 Algorithm/Model Analysis 26
3.4 Explanation of Different Modules 29
3.4.1 Document Capture Module 29
3.4.2 Character Recognition Module 30
3.4.3 Document Structure and Type Recognition Module 30
3.4.4 Schema-Constrained Extraction Module 31
3.4.5 Digitization API Module 33
3.4.6 Data Model 33

References **36**

Chapter 1

1 Introduction

Inventory record-keeping for small and medium retail businesses still depends heavily on paper documents. Every purchase from a supplier produces an invoice, receipt, or delivery slip, and each of these documents must currently be read by a person and entered manually into whatever inventory or accounting system the business uses. This process is slow, repetitive, and error-prone, and it is the actual bottleneck standing between a business and an accurate, up-to-date view of its stock and costs. Optical character recognition and large language models have both matured to the point where automating this reading step is technically realistic. However, using a language model to extract structured fields from a document introduces a reliability concern, since it can produce an incorrect or fabricated value without any clear indication that something has gone wrong. This project addresses that problem directly. The proposed system is a mobile-to-cloud document digitization gateway: a mobile application captures a document, a cloud service performs character recognition and document structure recognition, a language model extracts the document's content into a structured record under a fixed schema, and the result is delivered as a standardized JSON output through an API that an authorized inventory or ERP system can call. Section 1.1 states the problem, Section 1.2 gives its background, Section 1.3 sets out the research objectives, and Sections 1.4 through 1.6 cover the motivation, significance, and contributions of the work. The chapter closes with a complex engineering analysis in Section 1.7 and the organization of the report in Section 1.8.

1.1 Problem Statement

Small shops and fulfilment centres receive their purchase and sale records as unstructured documents, printed invoices, thermal-printer receipts, and handwritten chits,

issued by many different suppliers with no common layout. When one of these documents arrives, a person must read it and enter its contents manually, and this step is slow enough that records regularly fall behind or are skipped altogether. Existing inventory management systems do not solve this problem, since they assume the data entry has already been done by the time information reaches them and expect clean, structured input rather than a photograph of a paper slip. Standard OCR is not sufficient either, since it returns raw text without structure, so a downstream system still cannot tell which text is an item description, which is a quantity, or what type of document it is looking at in the first place. The engineering problem addressed by this project is recognizing both the characters and the structure of these heterogeneous supplier documents so that their content and document type can be identified automatically. The output is a standardized JSON representation of the document, delivered through an API that an ERP or inventory system can call to remove the manual data-entry step. Reconciling item names that differ across suppliers, such as "Coca-Cola" versus "Coke," is left to the consuming ERP or inventory system and is outside the scope of this project.

1.2 Problem Background

A large share of small and medium retail businesses still process supplier documents by hand: an employee reads each invoice or receipt and enters its contents into whatever system the business uses. This remains the case even though OCR and language models are, in principle, capable enough to automate the reading step. Two factors make this harder in practice. First, the input documents are heterogeneous. They arrive printed, handwritten, or thermal-printed, in layouts that differ from supplier to supplier, and a system must also determine the document's type before it can correctly interpret its content, since a purchase order is not structured the same way as a customer copy or a delivery note. Second, once a document's text and structure have been read, producing a clean, schema-valid record requires a language-model extraction step, and language models occasionally produce incorrect output without any visible indication that something is wrong. An extracted value therefore cannot

simply be trusted without some form of verification. Running this kind of processing reliably, at a speed and accuracy suitable for practical use, currently favors cloud-hosted models over on-device inference, since running a capable model directly on a mobile device introduces reliability, performance, and maintenance difficulties that a cloud-hosted approach avoids. For this reason, the mobile application is limited to capturing the document, while recognition and extraction take place on the server.

No existing category of software addresses this combination well. Commercial inventory and accounting software assumes that structured, typed input already exists by the time it reaches the system. General-purpose OCR services can read the text of a document but do not, on their own, determine document structure or type in the way a specific inventory workflow requires. Purpose-built invoice-extraction platforms, surveyed in Chapter 2, come closest, but none of the platforms reviewed treats document structure and type recognition as a first-class part of a capture-to-API pipeline in the way this project proposes. The problem this project addresses is therefore automated character and document-structure recognition for heterogeneous supplier documents, delivered as a JSON API that an existing inventory system can consume directly. Chapter 2 examines this gap against existing systems in more detail.

1.3 Research Objectives

The primary objective is to develop a mobile-to-cloud document digitization gateway that performs character recognition and document structure recognition on supplier invoices, receipts, and inventory documents, and outputs a standardized JSON representation via an API for consumption by authorized inventory and ERP systems. The supporting objectives are:

- To build a document recognition pipeline that combines character recognition (OCR) with document structure/type recognition, so that the system can both read the text of a heterogeneous set of supplier documents (printed and handwritten, English-language) and correctly identify what kind of document it is

looking at (e.g., purchase order, customer copy, delivery note, receipt).

- To design a schema-constrained extraction component in which a cloud-hosted language model converts recognized document content into structured line-item records conforming to a fixed JSON schema.
- To engineer a verification step around the language model's extraction output, cross-checking extracted numeric fields against the document's own arithmetic and flagging low-confidence records for review, so that no unverified value is silently finalized.
- To design the digitization service as a stateless, idempotent API: each submission (e.g., a mobile-app-scanned document) produces a JSON output without the system itself owning or persisting inventory state, while ensuring a retried or duplicate submission cannot produce duplicate or conflicting output to the downstream ERP/inventory system.
- To evaluate the system on character/structure recognition accuracy and extraction accuracy, including the rate at which arithmetic self-validation catches extraction errors.

1.4 Motivations

This project is motivated by three related observations.

The manual process is widespread and largely unautomated. Businesses that receive paper or photographed supplier documents overwhelmingly still enter their contents by hand, even when they already use digital inventory software, since that software assumes the structured input has already been produced by a person. Having a mobile app capture the document and a cloud service convert it directly into structured output addresses this bottleneck at its source, instead of simply adding another manual step further along.

Cloud-based recognition and extraction are now mature enough to make this practical, and the project's scope was kept accordingly nar-

row. Cloud OCR and language models are established commercial products (surveyed in Chapter 2), so document understanding does not need to be solved from first principles here. The engineering contribution of this project lies in combining character recognition, document structure recognition, and schema-constrained language-model extraction into a single pipeline; none of these individual components is being re-derived from scratch. On-device inference was also ruled out early on: our supervisor advised that running a capable model directly on a mobile device would add more difficulty than the project's scope requires.

Language models can produce incorrect output without any visible sign that something is wrong. This is the central problem on the extraction side of the pipeline: an unverified value from the model cannot simply be trusted. The system therefore checks extracted numeric fields against the document's own arithmetic and flags low-confidence results instead of returning them silently. Section 2.1.3 reviews the hallucination-detection and confidence-calibration literature that informed the design of this verification step.

1.5 Significance of the Research

The significance of this work is twofold.

Practically, the system removes the manual data-entry step between a supplier document being received and its contents becoming available in a structured, usable form, without requiring the business to replace its existing inventory or ERP software, since the output is delivered as a JSON API that these systems can call directly.

Technically, the project demonstrates one way to build automation around a language-model component that cannot be fully trusted: document structure and character recognition supply structured input to a schema-constrained extraction step, and arithmetic checks together with confidence scoring catch mistakes before the output is finalized, without requiring a human to review every submission. This pattern is not specific to inventory documents and could generalize to other settings where a language model extracts structured records from unstructured source

documents.

1.6 Research Contributions

The contributions of this project are as follows:

- **A character and document structure recognition pipeline** for heterogeneous, English-language supplier documents (printed and handwritten), including recognition of document type (e.g., purchase order, customer copy, delivery note, receipt) as a first-class output, not just raw text.
- **A schema-constrained extraction and verification layer**, in which an LLM converts recognized document content into structured line items conforming to a fixed JSON schema, with arithmetic self-validation and confidence scoring flagging low-confidence records rather than silently finalizing them.
- **A stateless, idempotent digitization API design**, in which the system does not own persistent inventory state but guarantees that a retried or duplicate document submission cannot result in duplicate or conflicting JSON output being delivered to a downstream ERP/inventory system.
- **An evaluation methodology** for such a system, covering character/structure recognition accuracy and extraction accuracy, including the rate at which arithmetic self-validation catches extraction errors.

1.7 Complex Engineering Analysis

This project is mapped to the Washington Accord Knowledge Profile (WK), Complex Engineering Problem (WP), and Complex Engineering Activity (EA) attributes required for a capstone project at this level. On the knowledge profile: evaluation metrics and confidence scoring apply mathematics and statistics (WK2); the mobile-to-cloud architecture, API design, and database design form the engineering fundamentals (WK3); NLP, language models, OCR, and document structure recognition constitute specialist, forefront knowledge (WK4); designing the API contract,

the document-type taxonomy, and the extraction and verification pipeline required deliberate engineering design decisions (WK5); cloud deployment practices, version control, and systematic testing reflect current engineering practice (WK6); responsible handling of business records and accountability for output correctness engage the engineer's role in society (WK7); and the overall design draws on the research literature in OCR, document understanding, structured LLM output, and hallucination mitigation (WK8).

1.7.1 Complex Engineering Problems

Table 1 maps the project to the complex engineering problem attributes.

Table 1: Complex Engineering Problem attributes

Attribute	**How the project satisfies it**
WP1: Depth of knowledge	Requires in-depth ML/NLP knowledge (OCR, document structure recognition) plus systems and API design, with sustained engagement with the research literature.
WP2: Conflicting requirements	Extraction automation vs. correctness; keeping the mobile client lightweight vs. giving the cloud service enough context to resolve document structure; usability for non-technical staff vs. system sophistication.
WP3: Depth of analysis	No off-the-shelf platform combines document-structure recognition with schema-constrained extraction tuned to heterogeneous, vendor-specific supplier documents; the recognition, verification, and evaluation methods must be designed, none having a closed-form answer.
WP4: Familiarity of issues	Reliable document-type recognition for small-business inventory documents is not a well-established, off-the-shelf combination in industry or research.
WP5: Applicable codes	Handling supplier/business documents engages data-handling and access-control expectations for the API (authorized-consumer access to the JSON output) and for any interim storage of submitted documents.
WP6: Stakeholder involvement	Shop/business staff submitting documents and the downstream ERP/inventory systems consuming the API form a stakeholder set with differing needs around speed, accuracy, and integration effort.
WP7: Interdependence	Character recognition, document structure recognition, and schema-constrained extraction are tightly coupled, since a structure-recognition error can misroute how downstream fields are extracted, so the sub-problems must be analyzed and tested jointly.

1.7.2 Complex Engineering Activities

Table 2 maps the project to the complex engineering activity attributes.

Table 2: Complex Engineering Activity attributes

Attribute	**How the project satisfies it**
EA1: Range of resources	Cloud OCR/document-AI services, language models, mobile app development tooling, and API infrastructure.
EA2: Level of interactions	Resolving interactions between document-structure recognition accuracy and the design of the extraction pipeline.
EA3: Innovation	Creative composition of existing recognition and language-model technologies into a document-capture-to-API pipeline for heterogeneous supplier documents, a combination not found as a first-class feature in the existing systems surveyed in Chapter 2.
EA4: Consequences to society	Lowers the data-entry burden for small and medium businesses, allowing existing inventory/ERP tooling to be automated without requiring a full software replacement.
EA5: Familiarity	Extends beyond prior coursework by applying principles-based approaches (system design under model uncertainty, API contract design, and evaluation methodology) to an open-ended, industry-relevant problem.

1.8 Thesis/Project Organization

Chapter 2 presents the background study: a literature review across four thematic areas, character and document structure recognition, schema-constrained language-model output, hallucination mitigation and confidence calibration, and document-gateway API and idempotency design, followed by a problem analysis that breaks down the technical challenges and compares the project against existing commercial systems. Chapter 3 presents the proposed research methodology, covering the system architecture, dataset and software requirements, the algorithm and model analysis behind each design choice, and the design of each pipeline module.

Summary

This chapter introduced the engineering problem addressed by the project: recognizing the characters and structure of heterogeneous supplier documents and extracting their content into a structured record that can be trusted. It set out the research objectives, the motivations behind the project, and its practical and technical significance, and listed four concrete contributions before mapping the project against the Complex Engineering Problem and Activity attributes expected of a capstone project at this level. Chapter 2 reviews the supporting literature and existing sys-

tems in more depth, and Chapter 3 presents the methodology by which the system is proposed to be built and evaluated.

Chapter 2

2 Background Study

Section 2.1 reviews the research literature the system design draws on, organized into four thematic areas corresponding to the system's pipeline stages: character and document structure recognition, schema-constrained language-model extraction, hallucination mitigation and confidence calibration, and API and idempotency design for a stateless service. Each subsection identifies the gap that remains between the surveyed work and the requirements of this project. Section 2.2 analyzes the problem itself, examines why existing commercial systems do not fully solve it, and consolidates the resulting research gap.

2.1 Literature Review

2.1.1 Character and Document Structure Recognition

Recognizing a document's text and its structural role is the first stage of the pipeline. Li et al. [1] introduced TrOCR, an end-to-end OCR architecture pairing a pretrained image Transformer encoder with a pretrained text Transformer decoder, which achieves strong results on printed receipts, handwritten text, and scene text without hand-built pre- or post-processing. Since this project is English-only, TrOCR's reported results, which are themselves on English and Latin-script benchmarks, apply directly, and its receipt benchmark corresponds closely to the printed-document class this project targets. Beyond reading text, the pipeline must also determine document structure and type, distinguishing, for example, a purchase order from a customer copy or a line-item table. *(Note: the following two citations are drawn from web-sourced summaries and still need to be verified directly against the original papers before this section is finalized.)* The LayoutLM family of models [2] extends a BERT-style architecture with two-dimensional position embeddings derived from OCR bounding-box coordinates, and later versions add visual features

directly, allowing the model to learn that spatially co-located text is often semantically related, such as a price sitting next to its line-item description. Reported document-image classification accuracy is around 90–95%. Kim et al. [3] proposed Donut, an OCR-free document understanding Transformer that reads a document image end-to-end without a separate text-recognition step, which avoids the error-propagation risk of a two-stage OCR-then-understanding pipeline. None of this work fully addresses the requirements of this project, however. TrOCR and similar OCR architectures are demonstrated on relatively clean, single-format benchmarks and do not themselves address document-type classification, and while LayoutLM and Donut address structure and type recognition, neither has been evaluated on the heterogeneous, vendor-specific invoice and receipt layouts this project targets. This gap is a key reason for evaluating document structure recognition directly on real supplier documents, in addition to reviewing published benchmark results.

2.1.2 Schema-Constrained Structured Output from Language Models

Converting recognized document content into the standardized JSON output requires the language model's output to conform to a fixed schema. Willard and Louf [4] reformulated guided generation as transitions over a finite-state machine, precomputing an index over the model's vocabulary so that invalid tokens can be masked with negligible overhead per step. This is the algorithm behind most practical constrained-decoding libraries, and it extends to context-free grammars through pushdown automata. Geng et al. [5] showed that grammar-constrained decoding allows a frozen, few-shot language model to match the performance of fine-tuned, task-specific models on structured NLP tasks. Geng et al. [6] introduced JSONSchemaBench, a benchmark of roughly ten thousand real-world JSON schemas evaluated across six constrained-decoding frameworks, and found measurable differences in efficiency and schema coverage between engines, which is directly relevant to selecting a decoding engine for this project's API output. Tam et al. [7], however, found that strict format restriction can degrade a model's reasoning performance

relative to free-form generation, with the degradation attributable to reasoning itself and not to parsing failures. This is a caution against relying on schema constraints alone to guarantee correct extraction. Constrained decoding guarantees only that the JSON output is syntactically valid, not that a given extracted value is correct; a schema-valid output can still contain an incorrect field value. This is the reason for adding a separate arithmetic-verification layer, described later in this chapter.

2.1.3 Hallucination Mitigation, Self-Verification, and Confidence Calibration

The verification layer draws on three related strands of research. Manakul et al. [8] proposed SelfCheckGPT, which samples multiple stochastic generations from a model and measures their mutual consistency to detect hallucination, without requiring access to the model's internals or an external database. This is directly applicable here, since the language model used for extraction runs behind a cloud API, where internal token probabilities are typically unavailable. Farquhar et al. [9] introduced semantic entropy, computed over clusters of semantically equivalent samples, which outperformed both naive entropy and self-evaluation baselines at predicting incorrect answers across several models and datasets. Ji et al. [10] showed that an iterative generate-score-refine self-reflection loop measurably reduces hallucination, providing a control-loop pattern that this project can adapt. On calibration, Guo et al. [11] showed that modern neural networks tend to be systematically overconfident, and that temperature scaling is a simple and largely effective post-hoc correction, while Kadavath et al. [12] showed that language models can produce reasonably calibrated self-assessments of their own answer correctness, with this ability improving as model scale increases. This body of work, however, is developed and evaluated on open-ended question answering and free-form generation. No prior work was found that specifically addresses confidence scoring for a structured field extracted from a supplier document, where the relevant ground truth is the document's own arithmetic and not an external fact. This is the gap the verification layer in this project is intended to address.

2.1.4 Document-Gateway Architecture and API Idempotency

The remaining design concern is architectural: how a document-digitization capability should be packaged so that an existing inventory or ERP system can consume it. The common industry pattern, sometimes described as a smart document service or a document-processing API gateway, separates capture from processing from consumption: a thin client uploads a document image, the recognition and extraction pipeline runs server-side, and the structured result is returned to an authorized system through a documented API rather than being written directly into that system's database. This separation allows a business to automate its data entry without replacing the inventory software it already uses, and it is the pattern adopted by this project. The commercial platforms surveyed in Section 2.2.2 show that this integration pattern is already standard, both for purpose-built extraction services and for inventory-side platforms.

Adopting this pattern introduces one specific obligation. Because the proposed system is a stateless gateway that does not own persistent inventory state, its correctness requirement is narrower than that of a system that does own such state, but it remains a genuine requirement: a duplicate document submission, for example a mobile upload retried after a dropped connection, must not result in duplicate or conflicting JSON output downstream, since the consuming ERP would otherwise double-count stock. Helland [13] establishes the general principle that in any system where an operation may be retried, message delivery can only be guaranteed as "zero or more times," never exactly once, and that message processing must therefore be designed to be idempotent. This is the reason each document submission carries a client-supplied identifier that the API can use to recognize and safely ignore a duplicate. One limitation of this literature is that it is written largely for systems that own persistent state, such as event-sourced ledgers or databases, while the smart-document-gateway pattern itself is documented mainly in product literature, not peer-reviewed work. The specific combination required here, an idempotent,

stateless recognition gateway that owns no inventory state, is not directly addressed by either body of work.

2.1.5 Summary of the Literature

Table 3 consolidates the reviewed literature by theme and the design decision each theme informs.

Table 3: Consolidated literature review by theme

Theme	**Key works**	**Informs**
Character & document structure recognition	TrOCR [1]; LayoutLM family [2]; Donut [3]	OCR stage architecture and document-type/structure classification
Schema-constrained output	FSM-guided decoding [4]; grammar-constrained LLMs [5]; JSONSchemaBench [6]; format-restriction caution [7]	JSON output stage; choice of cloud-side decoding approach
Hallucination & calibration	SelfCheckGPT [8]; semantic entropy [9]; self-reflection [10]; calibration [11]; self-knowledge [12]	Verification of extracted fields before finalizing output
Document-gateway architecture / idempotency	Idempotence [13]; commercial gateway practice (Section 2.2.2)	Stateless capture-to-API packaging; idempotent submission design

2.2 Problem Analysis

This section decomposes the problem into its constituent technical challenges, examines why existing systems do not fully address them, and consolidates the research gap.

2.2.1 Decomposition of the Technical Problem

Going from a photographed supplier document to a correct, standardized JSON output can be separated into three sub-problems, each with a distinct failure mode.

(1) Recognition under heterogeneity. Documents vary along two axes: their production method (printed, thermal, or handwritten) and their layout, since no two suppliers share a common template. The system must also determine a document's type and structure before its content can be interpreted correctly. Template-based

parsing is not viable given this layout variation, so recognition must be format-agnostic and must explicitly classify document structure and type instead of simply extracting raw text.

(2) Structured extraction, schema conformance, and verification. Mapping recognized text onto typed fields, such as item, quantity, unit price, line total, and document type, is an inference task performed by a language model. Constrained decoding [4] eliminates malformed output, but syntactic validity does not guarantee that the extracted values are correct; a well-formed JSON record can still contain an incorrect field value. This is addressed by checking extracted numeric fields against the document's own arithmetic, comparing quantity multiplied by unit price against the line and document totals, and flagging records that fail this check as low-confidence, instead of treating every extracted field as equally trustworthy.

(3) Reliable, idempotent API delivery without owned persistent state. Because the system does not own or persist inventory state, its correctness obligation is narrower than that of a system that owns a ledger, but it is not eliminated: every submission must produce its JSON output reliably, and a retried or duplicate submission must not cause duplicate or conflicting output to reach the ERP or inventory system that actually owns the inventory data.

2.2.2 Analysis of Existing Systems

Following the supervisor's guidance that the review should examine existing software alongside the academic literature, this section surveys the commercial landscape in three categories. Each category addresses part of the problem from a different angle, and none covers the full combination this project targets.

(a) General-purpose cloud document-AI platforms. AWS Textract, Google Document AI, and Azure AI Document Intelligence provide strong character recognition and, to varying degrees, field and table extraction for invoices and receipts. All three are cloud APIs and could plausibly serve as a foundation for a system such as this one, not necessarily as a competitor to it. Their accuracy differs on the subtask that matters most here: independent benchmarks report Azure and AWS as

stronger than Google Document AI specifically on invoice line-item extraction, with Google reported at roughly 82% field accuracy but noticeably weaker on line-item structure. None of the three platforms treats document-type classification, distinguishing a purchase order from a customer copy or delivery note, as a first-class output in the way this project does.

(b) Purpose-built invoice and receipt extraction APIs. Nanonets, Mindee, Veryfi, Docsumo, Affinda, and Klippa extend beyond the general platforms by offering invoice-specific field extraction together with a human-review interface for low-confidence fields, a pipeline shape reasonably close to this project's own extraction-and-verification stage. These platforms are useful points of comparison for the extraction sub-problem, but they remain general document-processing tools, not a document-structure-recognition-first pipeline built around the specific supplier-document taxonomy, purchase order, customer copy, delivery note, and receipt, that this project targets.

(c) Inventory and ERP-side platforms. Finale Inventory, HandiFox, and Apptricity already support mobile-capture-to-cloud-to-API integration for supplier invoices, typically routing a captured document through OCR and an approval step before pushing the result into an accounting or inventory backend. Their existence indicates that the mobile-to-cloud-to-API architecture adopted here is an established, low-risk pattern within this product category, not a novel one. These platforms, however, treat document capture as one step within a larger inventory system they own, and not as a standalone, stateless gateway that a business could call from whatever system it already uses.

Table 4 summarizes this comparison against the requirements derived in Section 2.2.1.

2.2.3 Consolidated Research Gap

Taken together, the literature review and the systems analysis point to a specific gap. Between the research reviewed and the commercial systems surveyed, the individual components needed for this project already exist: recognition architectures

Table 4: Comparison against existing commercial systems

Requirement	**General cloud document-AI**	**Purpose-built invoice APIs**	**Inventory/ERP platforms**	**Proposed system**
Reads printed and handwritten documents	Partial	Partial	No (assumes input)	Yes
Classifies document type (PO vs. receipt vs. customer copy vs. delivery note)	Partial	Partial	No	Yes
Recognizes document structure (line-item table regions)	Partial	Yes	No	Yes
Arithmetic self-validation of extracted fields before finalizing output	Partial	Yes	No	Yes
Mobile-capture-to-cloud gateway integration pattern	Partial (API only)	Yes	Yes	Yes
Delivered as a stateless JSON API for existing ERP/inventory systems, with no owned inventory state	Yes	Yes	Partial (owns inventory state)	Yes

for heterogeneous documents (Section 2.1.1), decoding constraints that guarantee schema validity (Section 2.1.2), hallucination detection and calibration techniques (Section 2.1.3), and an established mobile-capture-to-cloud-API pattern (Section 2.2.2). However, no academic work or commercial product reviewed for this study combines document structure and type recognition as a first-class, verified output with a stateless, idempotent capture-to-API pipeline built specifically for the heterogeneous, vendor-specific supplier documents that small and medium businesses receive. This combination is what the proposed system aims to provide. The contribution of this project therefore has two parts: composing existing recognition and extraction techniques into this specific pipeline, and evaluating its recognition and extraction accuracy on real supplier documents. The methodology by which this is proposed to be built and evaluated is presented in the next chapter.

Summary

This chapter reviewed the literature underlying each stage of the proposed pipeline, character and document structure recognition, schema-constrained output, hallucination mitigation, and document-gateway architecture, and, following the supervisor's guidance, examined the existing commercial landscape alongside it: general cloud document-AI platforms, purpose-built invoice extraction APIs, and inventory and ERP-side platforms. The problem analysis decomposed the technical challenge into three sub-problems and identified a specific research gap: no work reviewed, academic or commercial, combines document-structure recognition as a first-class output with a verified, stateless, idempotent document-capture-to-API pipeline for small-business inventory documents. Chapter 3 presents the research methodology proposed to address this gap.

Chapter 3

3 Research Methodology

Section 3.1 presents the proposed framework, covering the development approach and system architecture. Section 3.2 covers the dataset and software project requirements. Section 3.3 analyzes the algorithms and models underlying each design choice, including several decisions that remain open. Section 3.4 explains each of the six pipeline modules.

The work described in this chapter is proposed, not completed. This report is submitted at the design stage of the project: the framework, dataset plan, algorithm analysis, and module designs below describe what is intended to be implemented, while the implementation itself and its evaluation are the subject of Chapter 4. Where a design decision remains genuinely open, this chapter states the decision, the options under consideration, and the currently preferred choice, rather than presenting it as settled.

3.1 Proposed Framework

Section 1.1 stated the engineering problem as recognizing the characters and structure of a heterogeneous supplier document and extracting its content into a schema-valid structured record. Recognition is treated here as a perception task. Established architectures and accuracy metrics already exist for it, so the main design question is which components to combine and how to evaluate them on documents that are less controlled than the benchmarks those components were originally validated on. Extraction is a different kind of problem. It is an inference task performed under uncertainty, where the model's output cannot simply be accepted at face value, so the corresponding design question is how to limit the rate of silently incorrect output using evidence already present in the document.

Development approach. The project follows an iterative, evaluation-driven

development lifecycle, an Agile-style adaptation of the classical software development life cycle in which the recognition pipeline is built and refined across short, measurable cycles instead of in a single waterfall pass. The methodology proceeds in five phases: requirement analysis and document collection; data model and API contract design; the recognition pipeline; verification and the digitization API core; and evaluation and benchmarking, as shown in Figure 3.1. These phases are not strictly sequential. Results from the evaluation phase in Chapter 4 are expected to feed back into the API contract and the extraction design, with the pipeline refined across more than one cycle.

This iterative approach was chosen because several of the central design variables in this system do not have a closed-form answer. The confidence threshold that flags a low-confidence extraction for review, for instance, trades a false-positive rate against a silent-error rate, and the correct operating point cannot be derived analytically; it must be measured on real documents and tuned accordingly. The same applies to the JSON output schema, which cannot be finalized until the corpus reveals which fields actually appear across vendors, and to the document-type taxonomy, which requires real examples of each type before it can be fixed. A waterfall approach would require these decisions to be made when the least information is available. The feedback loop shown in Figure 3.1 exists to allow them to be revised as evidence accumulates, and the schema and taxonomy presented in Section 3.4 are treated as provisional, expected to be revised at least once after the corpus described in Section 3.2 has been collected.

System architecture. The proposed architecture is shown in Figure 3.2. A mobile application captures a supplier document as a photograph or scan and uploads it to a cloud endpoint. On the server side, the document passes through recognition (OCR and document structure and type classification), schema-constrained extraction, and arithmetic self-validation. The result is a standardized JSON representation that an authorized ERP or inventory management system can retrieve through the digitization API.

Two architectural boundaries constrain every module design in Section 3.4 and

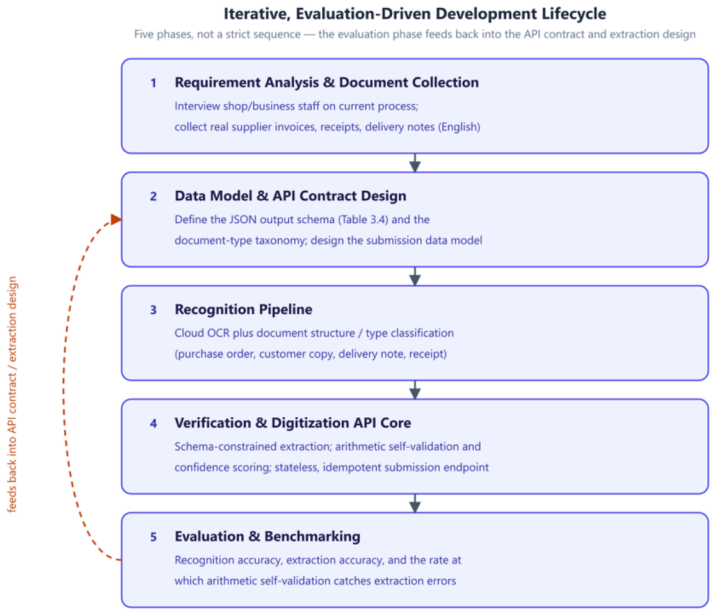

Figure 3.1: Development methodology: the phases of the iterative, evaluation-driven lifecycle and the feedback loop from evaluation back into the API contract and extraction design.

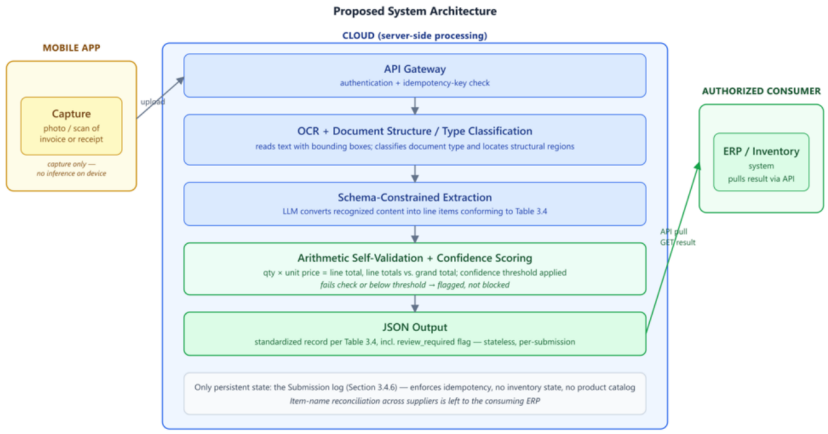

Figure 3.2: Proposed system architecture: mobile capture, cloud-hosted recognition and extraction pipeline with arithmetic self-validation, and delivery to an authorized ERP or inventory system via the digitization API.

are stated explicitly here.

The mobile client is capture-only. No recognition, extraction, or language-model inference runs on the device; all processing is cloud-hosted. The client's responsibilities are limited to image acquisition, basic capture-quality checks, idempotency-key generation, and upload. This follows the reasoning given in Section 1.2: running a capable language model on a mobile device introduces reliability, performance, and maintenance problems that a cloud-hosted deployment avoids, and the gap between what a phone can comfortably run and what reliable extraction over noisy documents requires is large enough that on-device inference would compromise the project's primary objective.

The system owns no inventory state. This is a digitization gateway, not an inventory management system, so it does not maintain stock levels, valuations, transaction history, or a product catalog; these remain the responsibility of the consuming ERP. The only persistent state maintained is operational: a submission log, described in Section 3.4, that exists to enforce idempotency. This boundary is what allows the system to be adopted without replacing software a business already runs, and it narrows the correctness obligation to the one described in Section 2.1.4, that a retried submission must not produce duplicate downstream output.

Figure 3.3 expands Figure 3.2 into a stage-by-stage view, showing the input and output of each module and the point at which the pipeline branches into arithmetic self-validation and low-confidence flagging.

3.2 Data Set Analysis / Software Project Requirement

3.2.1 Data Set Analysis

The corpus described in this section has not yet been collected. What follows is the collection and annotation protocol proposed for it.

Rationale. The system must be evaluated on documents representative of what it will actually encounter in practice. Public document-understanding benchmarks are useful but not sufficient on their own, since they tend to be relatively clean and

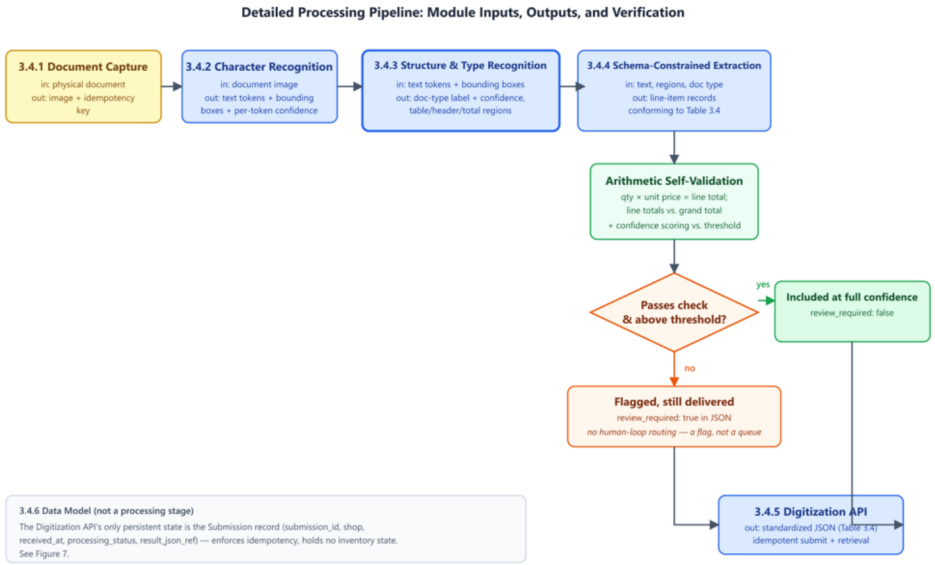

Figure 3.3: Detailed pipeline diagram: per-stage inputs and outputs from captured image through to JSON response, showing where arithmetic self-validation and confidence flagging attach.

do not include the thermal-printed and handwritten formats common in small-shop practice, which is exactly what this project's document-structure recognition needs to handle.

Collection protocol. Small shops and fulfilment centres will be approached and asked to contribute supplier documents they have already received. Participation will be voluntary and based on informed consent, with the purpose of the research, the intended use of the documents, and the retention period explained in advance. Documents will be captured through the same mobile capture path the finished system will use, so that image characteristics match real deployment conditions instead of idealized scans.

Target composition. The corpus is stratified along two axes, since both affect recognition difficulty: production method (printed, thermal-printed, handwritten) and document type (purchase order, customer copy, delivery note, receipt). The target is a minimum of 30 documents per document-type class to support a meaningful confusion matrix in Chapter 4, with the printed, thermal, and handwritten split reflecting natural prevalence instead of being forced to a uniform distribution, since a corpus that over-represents handwritten documents would give a misleading picture of real-world accuracy. Table 5 summarizes the target composition.

Table 5: Target corpus composition (provisional)

Document type	Printed	Thermal-printed	Handwritten	Min. total
Purchase order	majority	—	some	30
Delivery note	some	some	majority	30
Customer copy	majority	some	some	30
Receipt	some	majority	some	30

Annotation. Two layers of ground truth will be produced: a document-type label for each document, and a field-level transcription of the line items (description, quantity, unit price, line total) together with document-level totals.

Annotation will be carried out by team members against a written guideline. A subset of documents will be double-annotated and inter-annotator agreement reported, since evaluating against ground truth of unmeasured reliability would overstate the precision of the results.

Supplementary public data. Public benchmarks such as SROIE, CORD, and FUNSD will be used for pretraining and to sanity-check recognition performance against previously published figures.

Ethical considerations. Supplier documents contain commercially sensitive information, including prices, volumes, and trading relationships. Identifying details of contributing businesses and their suppliers will be anonymized in any published output of this research, document access will be limited to the project team, and contributors will be able to withdraw their documents. These commitments follow directly from the responsibilities identified in Section 1.7 (WK7 and WP5).

3.2.2 Software Project Requirement

Table 6 sets out the proposed technology stack. Several entries depend on the open decisions recorded in Section 3.3.

The evaluation harness is listed as a build requirement, not an afterthought, because the methodology is iterative and metrics are recomputed each cycle, which makes the measurement pipeline part of the system itself, even though its output is

Table 6: Proposed technology stack (provisional)

Layer	**Tool / technology**	**Purpose**	**Justification**
Mobile client	Cross-platform mobile framework	Document capture, quality checks, upload	Single codebase across platforms; the client is thin, so framework choice is low-risk
Backend service	Python web framework	API endpoints, pipeline orchestration	Python aligns with the ML/NLP tooling the pipeline depends on
OCR / document AI	Commercial cloud service	Text and layout recognition	Per Decision 1, Section 3.3
Language model	Commercial LLM API	Schema-constrained extraction	Per Decision 2, Section 3.3
Constrained decoding	Provider JSON mode or constrained-decoding library	Schema-valid output	Per Decision 3, Section 3.3
Datastore	Relational database	Submission log	The data model in Section 3.4 is relational and modest in scale
Version control	Git	Source management and collaboration	Standard practice
Testing	Unit/integration test framework; scripted evaluation harness	Correctness and reproducible metrics for Chapter 4	The evaluation methodology must be repeatable across iterations

reported only in Chapter 4.

3.3 Algorithm/Model Analysis

This section analyzes the algorithmic approach behind each stage of the pipeline and records the design decisions that remain open. Section 3.4 explains how these algorithms are composed into concrete modules.

Character recognition. The recognition stage is built on an existing cloud OCR service instead of a model trained from scratch, so the contribution of this project lies in the composition and the trust layer around the model, not in a new OCR architecture. TrOCR [1] is the architecture this design relies on: a Transformer encoder-decoder pairing pretrained vision and language models, which achieves strong results on printed, handwritten, and receipt-domain text without hand-built preprocessing. Bounding-box output is retained because the structure-recognition stage that follows depends on that spatial information. Donut [3], an OCR-free alternative, was considered and set aside, since adopting it would mean giving up mature commercial OCR services whose accuracy on real-world documents is well characterized, and it would require substantially more training data than the corpus plan in Section 3.2 is expected to produce.

Document structure and type classification. The same visual layout can mean different things depending on document type, so classification must occur before extraction. LayoutLM [2] is the relevant technique here, extending a text encoder with two-dimensional positional embeddings derived from OCR bounding boxes so that the model can learn that spatially co-located text is often related, with document-image classification among its demonstrated capabilities. Because the corpus does not yet exist, the classifier is staged in two steps: few-shot prompting of a language model over recognized text and a coarse layout description, which requires no training data but is more costly per document and less accurate, followed by a fine-tuned layout-aware classifier once the corpus described in Section 3.2 has been collected and labelled, evaluated against the few-shot baseline. This staged approach means that corpus collection does not block the remaining implementation work.

Schema-constrained extraction. Willard and Louf [4] showed that guided generation can be reformulated as a walk over a finite-state machine, with a precomputed vocabulary index so that invalid tokens can be masked at negligible cost per step. Geng et al. [5] showed that grammar-constrained decoding allows a few-shot model to match fine-tuned, task-specific models on structured tasks. JSONSchemaBench [6] found substantial differences between constrained-decoding engines in real-world schema coverage, which is the reason the specific decoding engine is left as an open decision instead of being fixed in advance. Constraining the output guarantees only that it is syntactically valid, not that its values are correct; a schema-valid record can still contain a hallucinated price. Tam et al. [7] further note that strict format restriction can degrade a model's reasoning relative to freer generation. Both findings motivate the separate verification step described below rather than relying on the schema alone.

Verification and confidence scoring. Two independent checks are applied to an extracted record before it is returned. The first is arithmetic self-validation: where quantity, unit price, and line total have all been extracted, quantity multiplied by unit price is checked against the line total directly, and line totals are summed and compared against the stated grand total. This check requires no model and adds negligible cost. The second is confidence scoring, in which each extracted record is scored and any record falling below a threshold is flagged as `review_required` in the output instead of being returned silently at full confidence. The reliability of the confidence signal itself is not assumed. Guo et al. [11] showed that modern neural networks tend to be overconfident and that temperature scaling is a simple post-hoc correction. Where model-internal probabilities are unavailable, as is common with commercial APIs, SelfCheckGPT [8] offers a black-box alternative of sampling multiple responses and measuring their consistency, with semantic entropy [9] providing a more principled clustering-based variant of the same idea.

Open algorithm and model decisions. Table 7 records the choices that remain open at this design stage.

Decision 4 is the most significant of these, since the confidence-scoring mechanism

Table 7: Open algorithm and model decisions

#	Decision	Options	Current preference and rationale
1	OCR / document-AI service	AWS Textract; Google Document AI; Azure AI Document Intelligence	Azure or AWS, on the line-item extraction benchmarking reported in Section 2.2.2; final choice pending a pilot on collected documents
2	Language model for extraction	Commercial LLM API providers	Undecided pending a pilot on collected documents; evaluated on schema-conformance and field accuracy
3	Constrained-decoding mechanism	Provider-native JSON mode; a library such as Outlines or XGrammar	Provider-native mode if schema coverage suffices, per JSONSchemaBench findings; library fallback otherwise
4	Confidence signal source	Token log-probabilities; sampled-consistency (SelfCheckGPT-style); a separately calibrated classifier	Undecided, and the biggest open question of the six. Log-probabilities are cheapest but often unavailable through commercial APIs; sampled consistency always works but multiplies inference cost
5	Document-type classifier	Few-shot LLM prompting; fine-tuned layout-aware model	Staged, as above: few-shot first, fine-tune after corpus collection
6	Result delivery	Synchronous response; asynchronous job with polling or webhook	Synchronous by default, since no human-confirmation step intervenes between submission and result; revisited if processing latency requires otherwise

depends on it entirely and the choice is further constrained by whichever provider is selected in Decision 2. It is scheduled for resolution during the first implementation cycle in Chapter 4, since it must be settled before the extraction-accuracy evaluation described there can be carried out.

3.4 Explanation of Different Modules

This section explains each of the six modules that compose the pipeline shown in Figure 3.3. For each module, the purpose, input, processing, output, and principal failure mode are stated; the algorithmic reasoning behind each module's processing was analyzed in Section 3.3 and is not repeated here.

3.4.1 Document Capture Module

Purpose. Acquire a usable image of a supplier document and submit it for processing.

Input. A physical document presented to the device camera, or an existing image or PDF from device storage.

Processing. The module performs capture-quality checks before upload, since recognition accuracy downstream depends on image quality and a rejected capture is far less costly than a silently misread document. These checks confirm that the full document is within frame, detect blur, and detect glare or low contrast, and if any check fails, the user is prompted to retake the photo instead of the failure passing unnoticed. Once a capture passes these checks, the module generates a client-supplied idempotency key, a UUID bound to the capture event rather than to the upload attempt, so that a retried upload after a network failure carries the same key as the original, and uploads the image together with that key.

Output. An image payload and its idempotency key, transmitted to the API gateway.

Failure mode. A poor-quality capture that nonetheless passes the quality checks will propagate recognition errors through the rest of the pipeline. This risk is reduced

rather than eliminated by the capture-quality checks; the arithmetic and confidence checks further downstream provide an additional layer of protection.

3.4.2 Character Recognition Module

Purpose. Convert the document image into machine-readable text with positional information.

Input. The uploaded document image.

Processing. A cloud OCR service performs text detection and recognition, using the architecture analyzed in Section 3.3, and returns recognized text tokens together with their bounding-box coordinates.

Output. Recognized text tokens with bounding boxes, and per-token recognition confidence where the OCR service provides it.

Failure mode. Misrecognized digits are the most damaging error class here, since an incorrect quantity or price is both consequential and difficult to detect by inspection. For this reason, Chapter 4 reports the character error rate on numeric fields separately from the overall figure.

3.4.3 Document Structure and Type Recognition Module

Purpose. Determine what kind of document is being processed and locate its structurally meaningful regions. This is the primary technical objective of the project.

Input. Recognized text tokens with bounding boxes from Section 3.4.2.

Processing. This module performs two related tasks using the staged approach analyzed in Section 3.3: classifying the document into one of the categories shown in Figure 3.4 (purchase order, customer copy, delivery note, or receipt), and identifying the structural regions of the page, namely the line-item table, header block, and totals block.

Output. A document-type label with a confidence score, together with bounding-box regions for the line-item table, header, and totals.

Failure mode. A misclassified document type is likely to cause every field extracted from that document to be misinterpreted, making this the most conse-

quential single failure mode in the pipeline. For this reason, Chapter 4 reports a full confusion matrix for document-type classification rather than accuracy alone.

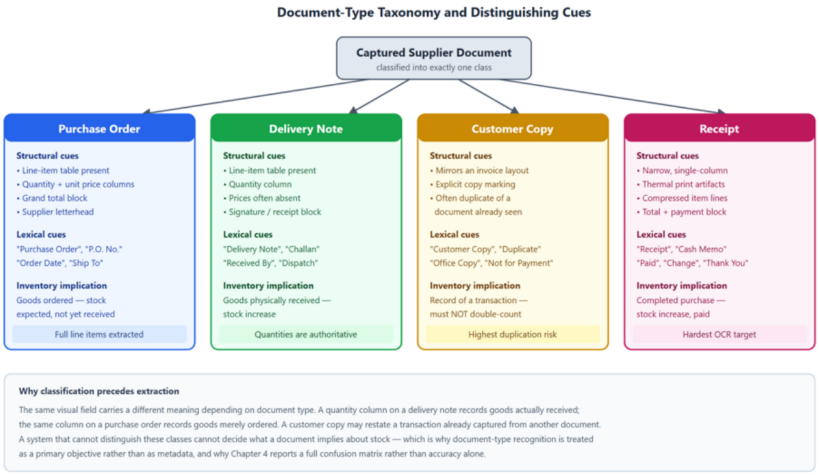

Figure 3.4: Document-type taxonomy: the four target classes with the structural and lexical cues that distinguish them, and the inventory implication of each.

3.4.4 Schema-Constrained Extraction Module

Purpose. Convert the recognized text of the identified regions into structured line-item records conforming to a fixed JSON schema.

Input. Recognized text, structural regions, and document type from Sections 3.4.2–3.4.3.

Processing. A cloud-hosted language model performs the extraction using the constrained-decoding approach analyzed in Section 3.3, which guarantees that the output conforms to the schema in Table 8.

Output. A set of line-item records conforming to the schema in Table 8.

Failure mode. A record may be well-formed and still contain an incorrect value. This is the residual failure mode that arithmetic self-validation and confidence scoring, described in Section 3.3, are designed to catch.

The `arithmetic_check` field is included directly in the API contract rather than kept as internal state, since a consuming ERP must be able to distinguish a value the system has verified from one it has merely extracted.

Table 8: Target JSON output schema (provisional; to be revised after corpus collection)

Field	Type	Req.	Description
`submission_id`	string (UUID)	Yes	Client-supplied idempotency key
`document_type`	enum	Yes	`purchase_order` | `customer_copy` | `delivery_note` | `receipt`
`document_type_confidence`	float	Yes	Classifier confidence, 0–1
`supplier_name`	string	No	As printed on the document
`document_date`	date (ISO 8601)	No	Document-issue date
`document_number`	string	No	Invoice or reference number
`line_items`	array	Yes	Zero or more line-item objects
`line_items[].raw_text`	string	Yes	Item description exactly as recognized
`line_items[].quantity`	number	No	Quantity as extracted
`line_items[].unit_price`	number	No	Unit price as extracted
`line_items[].line_total`	number	No	Line total as extracted
`line_items[].arithmetic_check`	enum	Yes	`passed` | `failed` | `not_applicable`
`grand_total`	number	No	Document total as extracted
`review_required`	boolean	Yes	True if any line item fails the arithmetic check or falls below the confidence threshold

3.4.5 Digitization API Module

Purpose. Expose the pipeline as a service that an authorized ERP or inventory system can consume.

Input. Submissions from the mobile client and result requests from consuming systems.

Processing. The API exposes a submission endpoint that accepts a document image and idempotency key, and a retrieval path that returns the standardized JSON. Consuming systems authenticate as clients scoped to a particular shop. Idempotency is enforced at the submission endpoint: Helland [13] shows that in any system where an operation may be retried, delivery can only be guaranteed as "zero or more times," never exactly once, so the service records every `submission_id` it has already processed, and a submission arriving with a previously seen key returns the original result instead of being reprocessed.

Output. A standardized JSON document conforming to Table 8.

Failure mode. Two genuinely distinct captures colliding on the same idempotency key would cause one of them to be silently discarded. To avoid this, the key is generated per capture event rather than per user action, and is bound to the image payload itself.

3.4.6 Data Model

Although the system holds no inventory state, it still requires a minimal amount of persistent state to operate. A single entity is proposed for this purpose: Submission, the idempotency and audit record referenced in Section 3.4.5, which holds the submission ID, shop, received timestamp, processing status, and a reference to the generated JSON result.

What is deliberately excluded from this model is as significant as what is included: there is no Shop-owned catalog, Stock, Transaction, or Valuation entity, since item-name reconciliation and inventory state both belong to the consuming ERP. A reviewer checking whether the architectural boundary described in Section

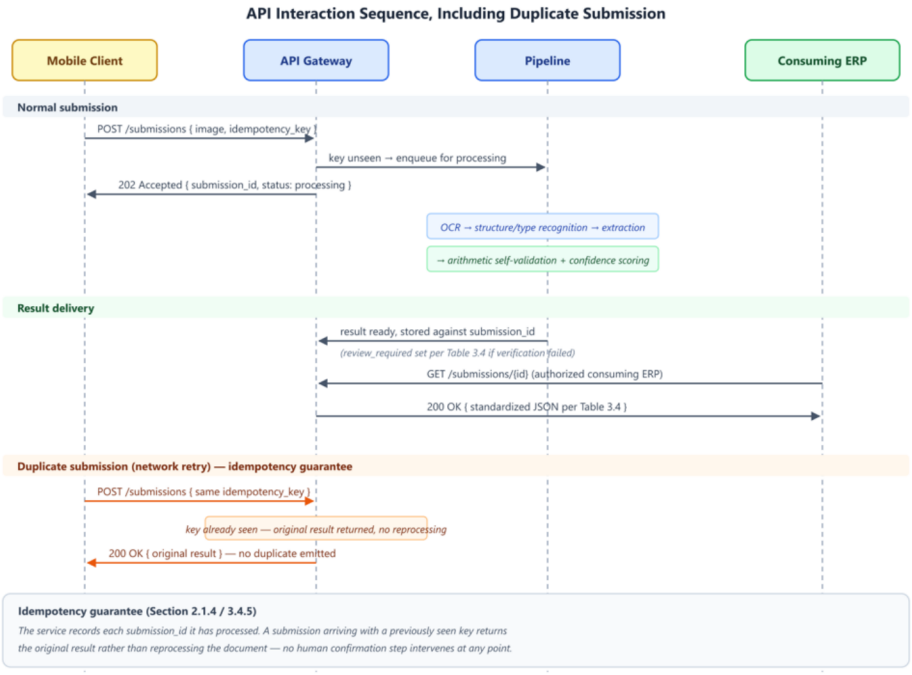

Figure 3.5: API sequence diagram: capture, submit (with idempotency key), process, result retrieval by the consuming ERP, including the duplicate-submission path returning the original result without reprocessing.

3.1 is genuinely respected can verify it against this data model.

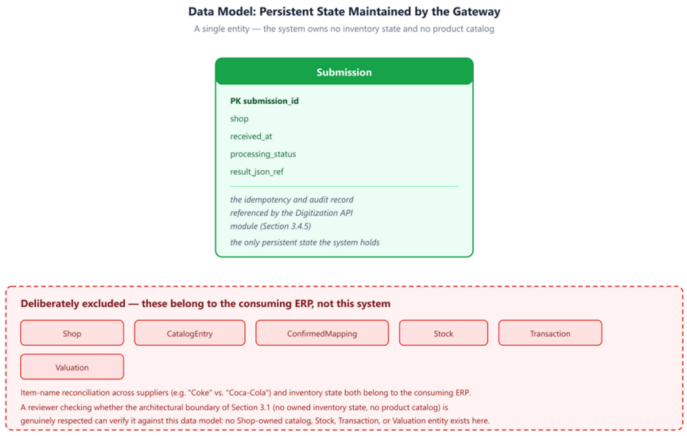

Figure 3.6: Entity-relationship diagram: the Submission record, alongside the inventory and catalog entities deliberately excluded from the gateway's scope.

Summary

This chapter presented the proposed research methodology. The framework combines an iterative, evaluation-driven development approach, chosen because several of the system's central design variables cannot be fixed correctly before real ev-

idence is available, with a system architecture constrained by two boundaries: a capture-only mobile client and a gateway that owns no inventory state. The dataset collection and annotation protocol was described, though it has not yet been carried out, along with the software project requirements. The algorithms underlying recognition, classification, extraction, and verification were analyzed, and six design decisions were recorded as open rather than presented as settled. Each of the six pipeline modules was then explained, together with its failure mode and the shared data model underlying them.

What has been presented in this chapter is a design rather than a result. Chapter 4 will report on the implementation of this methodology, the evaluation metrics and performance analysis it produces, and a comparison against the baselines identified throughout this chapter.

References

[1] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "Trocr: Transformer-based optical character recognition with pre-trained models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 13094–13102, 2023.

[2] Y. Xu *et al.*, "Layoutlm: Pre-training of text and layout for document image understanding," *KDD*, 2020. Citation to be verified directly against the source publication (and successors LayoutLMv2, LayoutLMv3) before finalizing; currently sourced from secondary/web summaries, not read directly.

[3] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, "Ocr-free document understanding transformer," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022. arXiv:2111.15664.

[4] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[5] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured nlp tasks without finetuning," in *Proceedings of EMNLP*, pp. 10932–10952, 2023.

[6] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "Jsonschemabench: A rigorous benchmark of structured outputs for language models," *arXiv preprint arXiv:2501.10868*, 2025.

[7] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let me speak freely? a study on the impact of format restrictions on performance of large language models," in *Proceedings of EMNLP: Industry Track*, pp. 1218–1236, 2024.

[8] P. Manakul, A. Liusie, and M. J. F. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," in *Findings of ACL: EMNLP 2023*, pp. 9004–9017, 2023.

[9] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, pp. 625–630, 2024.

[10] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating hallucination in large language models via self-reflection," in *Findings of ACL: EMNLP 2023*, pp. 1827–1843, 2023.

[11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of ICML, PMLR 70*, 2017.

[12] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, *et al.*, "Language models (mostly) know what they know," *arXiv preprint arXiv:2207.05221*, 2022.

[13] P. Helland, "Idempotence is not a medical condition," *Communications of the ACM*, vol. 55, no. 5, pp. 56–65, 2012.